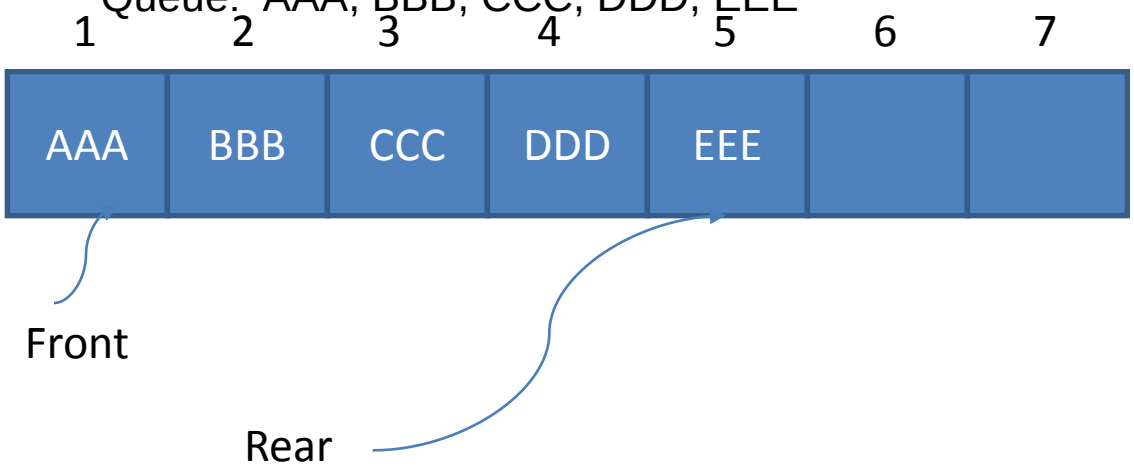
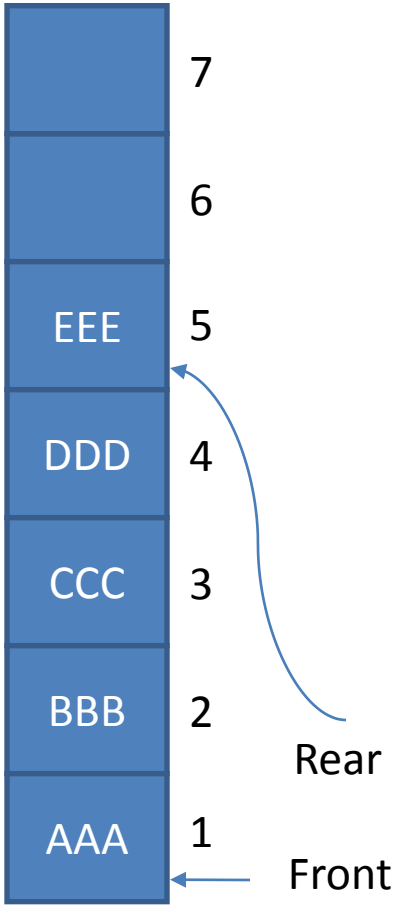
Queue

Queue (Linear Queue)

- It is a linear data structure consisting of list of items.
- In queue, data elements are added at one end, called ***the rear*** and removed from another end, called ***the front*** of the list.
- Two basic operations are associated with queue:
  1. "Insert" operation is used to insert an element into a queue.
  2. "Delete" operation is used to delete an element from a queue.
- FIFO list
- Example:

Queue: AAA, BBB, CCC, DDD, EEE





Algorithms for Insert and Delete Operations in Linear Queue

For Insert Operation

Insert-Queue(Queue, Rear, Front, N, Item)

Here, Queue is the place where to store data. Rear represents the location in which the data element is to be inserted and Front represents the location from which the data element is to be removed. Here N is the maximum size of the Queue and finally, Item is the new item to be added.

1. If Rear = N then Print: Overflow and Return. /*…Queue already filled..*/

2. Set Rear := Rear +1

3. Set Queue[Rear] := Item

4. Return.

For Delete Operation

Delete-Queue(Queue, Front, Rear, Item)

Here, Queue is the place where data are stored. Rear represents the location in which the data element is to be inserted and Front represents the location from which the data element is to be removed. Front element is assigned to Item.

1. If Front = N+1 then Print: Underflow and Return. /*…Queue Empty

2. Set Item := Queue[Front]

3. Set Front := Front + 1

4. Return.

Example: Consider the following queue (linear queue).

Rear = 4 and Front = 1 and N = 7

10	50	30	40			
1	2	3	4	5	6	7

(1) Insert 20. Now Rear = 5 and Front = 1

10	50	30	40	20		
1	2	3	4	5	6	7

(2) Delete Front Element. Now Rear = 5 and Front = 2

	50	30	40	20		
1	2	3	4	5	6	7

(3) Delete Front Element. Now Rear = 5 and Front = 3

		30	40	20		
1	2	3	4	5	6	7

(4) Insert 60. Now Rear = 6 and Front = 3

		30	40	20	60	
1	2	3	4	5	6	7

Drawback of Linear Queue

- Once the queue is full, even though few elements from the front are deleted and some occupied space is relieved, it is not possible to add anymore new elements, as the rear has already reached the Queue's rear most position.

Circular Queue

- This queue is not linear but circular.
- Its structure can be like the following figure:
- In circular queue, once the Queue is full the "First" element of the Queue becomes the "Rear" most element, if and only if the "Front" has moved forward. otherwise it will again be a "Queue overflow" state.

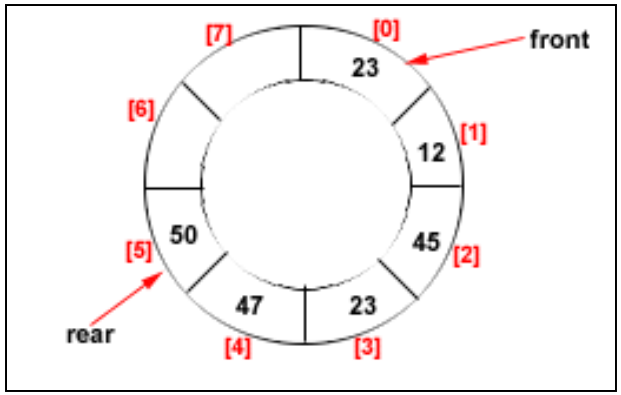


Figure: Circular Queue having Rear = 5 and Front = 0

Algorithms for Insert and Delete Operations in Circular Queue

For Insert Operation

Insert-Circular-Q(CQueue, Rear, Front, N, Item)

Here, CQueue is a circular queue where to store data. Rear represents the location in which the data element is to be inserted and Front represents the location from which the data element is to be removed. Here N is the maximum size of CQueue and finally, Item is the new item to be added. Initailly Rear = 0 and Front = 0.

1. If Front = 0 and Rear = 0 then Set Front := 1 and go to step 4.
2. If Front =1 and Rear = N or Front = Rear + 1

   then Print: “Circular Queue Overflow” and Return.
3. If Rear = N then Set Rear := 1 and go to step 5.
4. Set Rear := Rear + 1
5. Set CQueue [Rear] := Item.
6. Return

For Delete Operation

Delete-Circular-Q(CQueue, Front, Rear, Item)

Here, CQueue is the place where data are stored. Rear represents the location in which the data element is to be inserted and Front represents the location from which the data element is to be removed. Front element is assigned to Item. Initially, Front = 1.

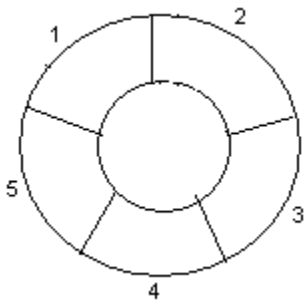
1. If Front = 0 then

   Print: “Circular Queue Underflow” and Return. /*..Delete without Insertion

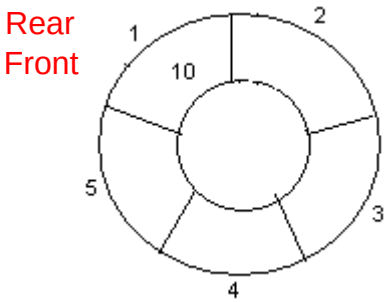
2. Set Item := CQueue [Front]
3. If Front = N then Set Front = 1 and Return.
4. If Front = Rear then Set Front = 0 and Rear = 0 and Return.
5. Set Front := Front + 1
6. Return.

Example: Consider the following circular queue with N = 5.

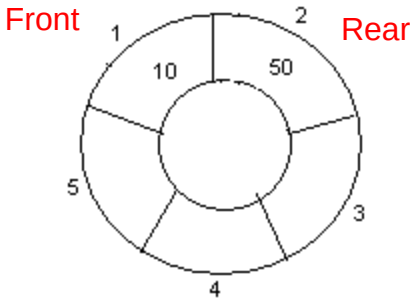
1. Initially, Rear = 0, Front = 0.



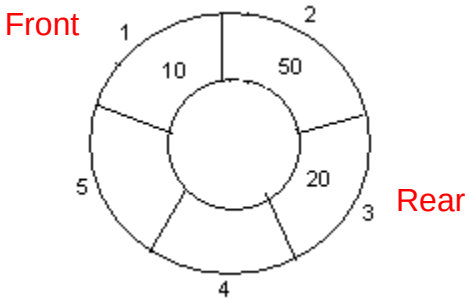
2. Insert 10, Rear = 1, Front = 1.



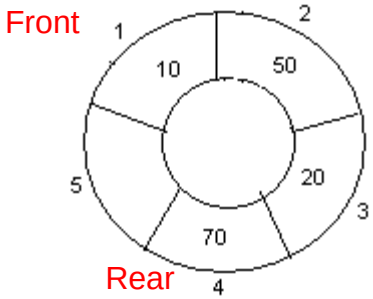
3. Insert 50, Rear = 2, Front = 1.



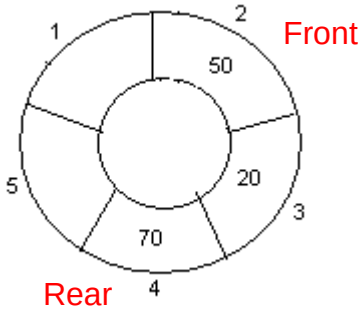
4. Insert 20, Rear = 3, Front = 0.



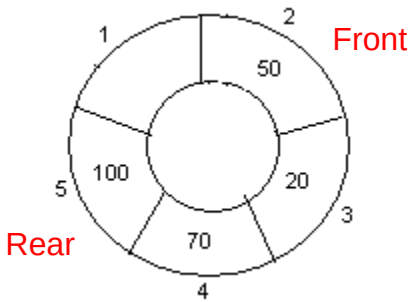
5. Insert 70, Rear = 4, Front = 1.



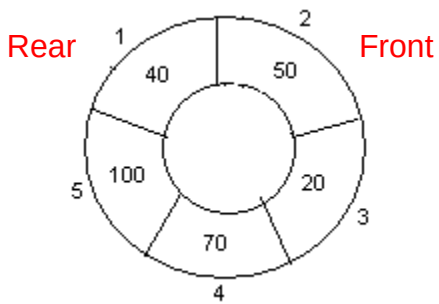
6. Delete front, Rear = 4, Front = 2.



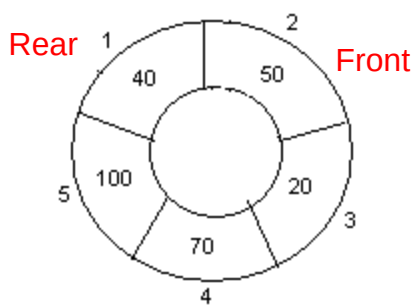
7. Insert 100, Rear = 5, Front = 2.



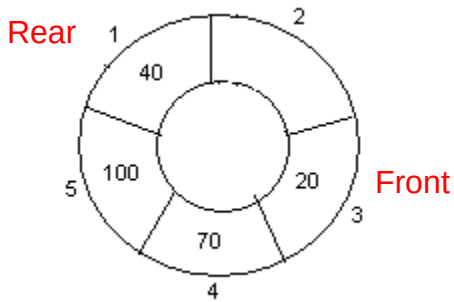
8. Insert 40, Rear = 1, Front = 2.



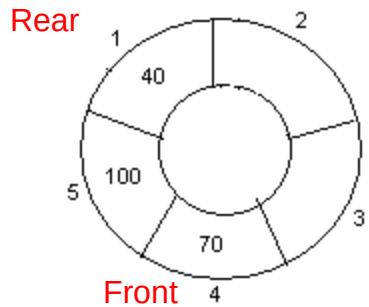
9. Insert 140, Rear = 1, Front = 2.
As Front = Rear + 1, so Queue overflow.



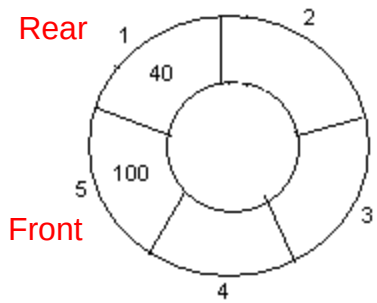
10. Delete front, Rear = 1, Front = 3.



11. Delete front, Rear = 1, Front = 4.



12. Delete front, Rear = 1, Front = 5.



Types of Queues in Data Structure

Queue in data structure is of the following types

- Simple Queue
- Circular Queue
- Priority Queue
- Dequeue (Double Ended Queue)

Simple Queue

The simple queue is a normal queue where insertion takes place at the **FRONT** of the queue and deletion takes place at the **END** of the queue.

Circular Queue

In a circular queue, the last node is connected to the first node.
Circular queue is also called as **Ring Buffer.**
Insertion in a circular queue happens at the **FRONT** and deletion at the **END** of the queue.

Priority Queue

In a priority queue, the nodes will have some predefined priority.
Insertion in a priority queue is performed in the order of arrival of the nodes.
The node having the least priority will be the first to be removed from the priority queue.

Dequeue (Doubly Ended Queue)

In a Double Ended Queue, insertion and deletion operations can be done at both **FRONT** and **END** of the queue.

Deque or Double Ended Queue is a generalized version of Queue data structure that allows insert and delete at both ends.

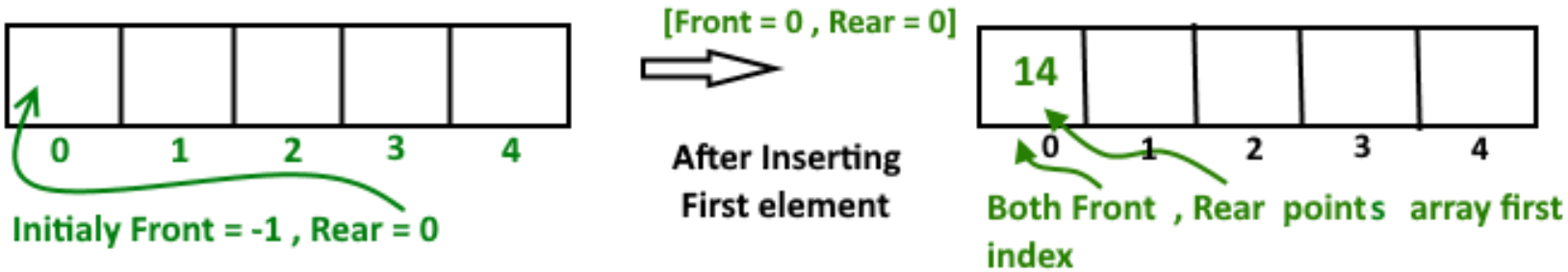
insertFront(): Adds an item at the front of Deque.

insertLast(): Adds an item at the rear of Deque.

deleteFront(): Deletes an item from front of Deque.

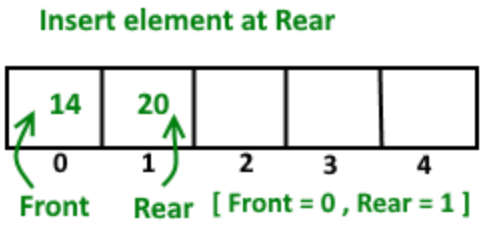
deleteLast(): Deletes an item from rear of Deque.

Circular array implementation deque



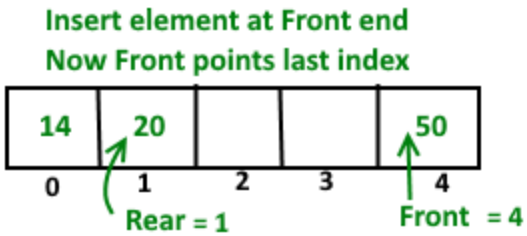
Insert Elements at Rear end

```
a). First we check deque if Full or Not
b). IF Rear == Size-1
     then reinitialize Rear = 0 ;
  Else increment Rear by '1'
     and push current key into Arr[ rear ] = key
Front remain same.
```



Insert Elements at Front end

```
a). First we check deque if Full or Not
b). IF Front == 0 || initial position, move Front to
points last index of array front = size - 1
    Else decremented front by '1' and push current key
into Arr[ Front] = key Rear remain same.
```

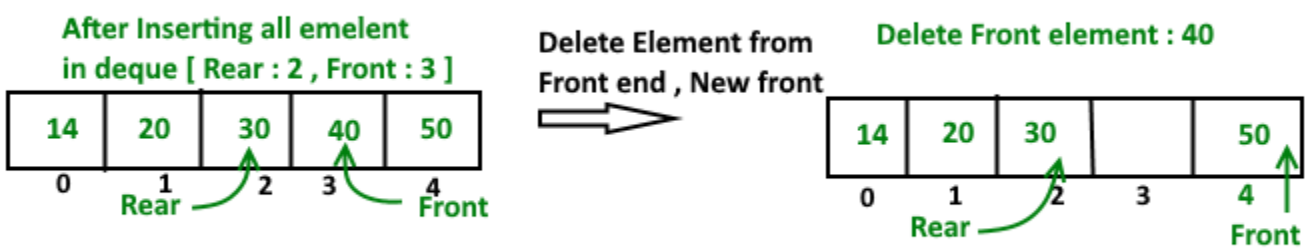


Delete Element From Rear end

```
a). first Check deque is Empty or Not
b). If deque has only one element
      front = -1 ; rear =-1 ;
   Else IF Rear points to the first index of array it's means we have to move rear to points last index
     [ now first inserted element at front end become rear end ]
      rear = size-1 ;
   Else || decrease rear by '1'
      rear = rear-1;
```

Delete Element From Front end

```
a). first Check deque is Empty or Not
b). If deque has only one element front = -1 ; rear =-1 ;
    Else IF front points to the last index of the array it's means we have no more elements in array
         so we move front to points first index of array
       front = 0 ;
    Else || increment Front by '1'
      front = front+1;
```



END!!!